

Algorithms Course: Unified Lecture Notes

Synthesized from Lectures 15–22

Fall 2025

Contents

1	Module 1: Graph Theory Fundamentals	3
1.1	Basic Concepts	3
1.2	Graph Representations	3
2	Module 2: Graph Traversal	4
2.1	Depth First Search (DFS)	4
2.2	Breadth First Search (BFS)	4
3	Module 3: Minimum Spanning Trees (MST)	5
3.1	Kruskal's Algorithm (Global Optimization)	5
3.1.1	Logic	5
3.2	Prim's Algorithm (Local Optimization)	5
3.2.1	Logic	5
3.3	Comparison	6
4	Module 4: Matrix Multiplication	7
4.1	Approaches	7
4.2	Strassen's Algorithm	7
5	Module 5: Polynomials and the Fast Fourier Transform (FFT)	8
5.1	The Challenge: Representation Matters	8
5.2	The Fast Fourier Transform (FFT)	8
5.2.1	Divide and Conquer Strategy	8
5.2.2	Roots of Unity	8
5.2.3	The Butterfly Operation (Visualization)	9
5.3	FFT Algorithm Implementation	9
5.4	Inverse FFT and Complexity	9
6	Module 6: String Matching	10
6.1	Naive Approach	10
6.2	Knuth-Morris-Pratt (KMP) Algorithm	10
6.2.1	The Failure Function (Next Array)	10
6.2.2	KMP Algorithms	10
6.2.3	Complexity Analysis	10

1 Module 1: Graph Theory Fundamentals

Source: Lecture 15

This module introduces the basic structures used to represent relationships between objects.

1.1 Basic Concepts

- **Definition:** A graph $G = (V, E)$ consists of a set of vertices V and a set of edges E .
- **Visualization:** A diagram of nodes and lines is merely a *representation* or drawing of a graph, not the graph itself. The same graph can be drawn in multiple ways.
- **Simple Graphs:** The course focuses on simple graphs, which contain:
 - No self-loops.
 - No multiple edges between the same pair of vertices.
- **Planar Graphs:** A graph is planar if it can be drawn such that no edges intersect. Example: The complete graph with 4 vertices (K_4) is planar.

1.2 Graph Representations

The choice of representation affects the space complexity and the time complexity of graph operations.

Representation	Description	Space	Check (u, v)	Traverse Neighbors
Adjacency Matrix	A 2D array A where $A[i][j] = 1$ if an edge exists, else 0.	$O(V ^2)$	$O(1)$	$O(V)$
Adjacency List	An array where each index u points to a linked list of neighbors.	$O(V + E)$	$O(\deg(u))$	$O(\deg(u))$
Edge List	A simple list of all pairs (u, v) in E .	$O(E)$	$O(E)$	$O(E)$

Table 1: Comparison of Graph Representations

Usage Note: Adjacency matrices are preferred for dense graphs, while adjacency lists are preferred for sparse graphs to conserve space.

2 Module 2: Graph Traversal

Sources: Lecture 16, 18, 19

Traversal algorithms visit all vertices in a graph systematically.

2.1 Depth First Search (DFS)

DFS explores as deep as possible along each branch before backtracking.

- **Algorithm Logic:**

1. Mark the current vertex v as visited.
2. Perform “Pre-Work” on v .
3. For every edge (v, w) :
4. If w is unmarked, recursively call $\text{DFS}(G, w)$.
5. Perform “Post-Work” on the edge or vertex.

- **Time Complexity:** $O(|V| + |E|)$.

- **Timestamps:** We can track discovery time (DFS_NO) to establish an ordering:

$$v.\text{DFS} = \text{DFS_NO}; \quad \text{DFS_NO} = \text{DFS_NO} + 1$$

- **Edge Classification:** DFS builds a tree structure.

- **Tree Edges:** Edges (u, v) where DFS discovered v via u .
- **Back Edges:** Edges connecting a vertex to one of its ancestors in the DFS tree.

- **Connectivity:**

- If the graph is connected, DFS visits all vertices.
- **Strongly Connected:** Directed graph where every vertex is reachable from every other vertex.
- **Weakly Connected:** Connected if edge directions are ignored.

2.2 Breadth First Search (BFS)

BFS explores all neighbor nodes at the present depth prior to moving on to the nodes at the next depth level.

- **Algorithm Logic:**

1. Use a **Queue** data structure.
2. Start by marking the source vertex v and putting it in the Queue.
3. While the Queue is not empty:
 - Remove vertex w .
 - For all edges (w, x) :
 - If x is unmarked: Mark x , add edge (w, x) to tree T , and enqueue x .

- **Application:** Finding the shortest path in unweighted graphs.

3 Module 3: Minimum Spanning Trees (MST)

Sources: Lecture 18, Lecture 22

Definition: Given a connected, undirected, weighted graph G , an MST is a subgraph that connects all vertices with the minimum possible total edge weight and contains no cycles.

3.1 Kruskal's Algorithm (Global Optimization)

Kruskal's algorithm builds the MST by treating the graph as a forest of trees and merging them. It focuses on the edges, selecting the smallest weight edge available globally that does not form a cycle.

3.1.1 Logic

1. **Sort** all edges in non-decreasing order of their weights.
2. **Iterate** through the sorted edges. For each edge (u, v) :
3. **Check** if u and v belong to different sets (using the **Union-Find** data structure).
4. **Union:** If they are in different sets, add (u, v) to the MST and merge the sets. If they are in the same set, adding the edge would create a cycle, so ignore it.

Algorithm 1 Kruskal's Algorithm

```
1:  $A \leftarrow \emptyset$ 
2: for each vertex  $v \in V$  do
3:   Make-Set( $v$ )
4: end for
5: Sort the edges of  $E$  by non-decreasing weight  $w$ 
6: for each edge  $(u, v) \in E$ , taken in sorted order do
7:   if Find-Set( $u$ )  $\neq$  Find-Set( $v$ ) then
8:      $A \leftarrow A \cup \{(u, v)\}$ 
9:     Union( $u, v$ )
10:  end if
11: end for
12: return  $A$ 
```

Complexity: $O(E \log E)$ or $O(E \log V)$ (dominated by sorting the edges).

3.2 Prim's Algorithm (Local Optimization)

Prim's algorithm builds the MST by growing a single tree from a starting vertex. At each step, it adds the cheapest edge connecting a vertex in the tree to a vertex outside the tree.

3.2.1 Logic

- Maintain a priority queue Q of vertices outside the tree, keyed by the minimum weight edge connecting them to the tree.
- **Key Property:** $key[v]$ is the minimum weight of any edge (u, v) where $u \in MST$.
- Extract the minimum vertex, add it to MST, and update the keys of its neighbors (Relaxation).

Algorithm 2 Prim's Algorithm

```
1: Input: Graph  $G(V, E)$ , weights  $w$ , start vertex  $r$ 
2: for each  $u \in V$  do
3:    $key[u] \leftarrow \infty$ 
4:    $parent[u] \leftarrow \text{nil}$ 
5: end for
6:  $key[r] \leftarrow 0$ 
7:  $Q \leftarrow V$  ▷ Priority Queue containing all vertices
8: while  $Q \neq \emptyset$  do
9:    $u \leftarrow \text{Extract-Min}(Q)$  ▷ Vertex with lightest connection to tree
10:  for each neighbor  $v$  of  $u$  do
11:    if  $v \in Q$  and  $w(u, v) < key[v]$  then
12:       $parent[v] \leftarrow u$ 
13:       $key[v] \leftarrow w(u, v)$ 
14:       $\text{Decrease-Key}(Q, v, w(u, v))$ 
15:    end if
16:  end for
17: end while
```

Complexity:

- Using Binary Heap: $O(E \log V)$.
- Using Fibonacci Heap: $O(E + V \log V)$.

3.3 Comparison

Feature	Kruskal's	Prim's
Approach	Process edges (Global)	Process vertices (Local)
Data Structure	Union-Find	Priority Queue
Preferred Graph	Sparse Graphs ($E \ll V^2$)	Dense Graphs ($E \approx V^2$)
Tree Growth	Grows a forest that merges	Grows a single tree

Table 2: Kruskal vs. Prim

4 Module 4: Matrix Multiplication

Sources: Lecture 19, 20

4.1 Approaches

- **Standard Algorithm:** Requires $O(n^3)$ operations.
- **Divide and Conquer:**
 - Split matrices A and B into four $n/2 \times n/2$ sub-matrices.
 - C_{ij} is calculated using 8 multiplications of these sub-matrices.
 - Recurrence: $T(n) = 8T(n/2) + O(n^2)$, which results in $O(n^3)$.

4.2 Strassen's Algorithm

Strassen optimized the Divide and Conquer approach to reduce the number of recursive multiplications from 8 to 7.

- **Logic:** Construct 7 intermediate matrices (P_1 to P_7) using sums/differences of the sub-matrices.

Example: $P_1 = a(f - h)$, $P_2 = (a + b)h, \dots$

- **Complexity:**

$$T(n) = 7T(n/2) + O(n^2)$$

- **Solution:** Using the Master Theorem, $T(n) = O(n^{\log_2 7}) \approx O(n^{2.81})$.
- **Winograd's Variant:** A further optimization of Strassen's method to reduce constant factors.

5 Module 5: Polynomials and the Fast Fourier Transform (FFT)

Sources: Lecture 20, 21

Goal: Efficiently multiply two polynomials of degree $n - 1$.

5.1 The Challenge: Representation Matters

A polynomial $P(x) = \sum_{i=0}^{n-1} a_i x^i$ can be represented in two ways:

1. **Coefficient Form:** The vector of coefficients $a = [a_0, a_1, \dots, a_{n-1}]$.
 - Addition: $O(n)$
 - Multiplication (Convolution): $O(n^2)$
2. **Point-Value Form:** A set of n point-value pairs $\{(x_0, y_0), \dots, (x_{n-1}, y_{n-1})\}$.
 - Addition: $O(n)$
 - Multiplication: $O(n)$ (Simply multiply values at corresponding points: $C(x_k) = A(x_k) \cdot B(x_k)$).

Strategy: To multiply fast ($O(n \log n)$), we convert from Coefficient $\rightarrow$ Point-Value (using FFT), multiply in $O(n)$, and convert back (Inverse FFT).

5.2 The Fast Fourier Transform (FFT)

The FFT is a Divide and Conquer algorithm to evaluate a polynomial at n complex roots of unity.

5.2.1 Divide and Conquer Strategy

We split the polynomial $P(x)$ into even and odd indexed coefficients:

$$P(x) = (a_0 + a_2x^2 + \dots) + x(a_1 + a_3x^2 + \dots)$$

Let $P_{\text{even}}(y) = a_0 + a_2y + \dots$ and $P_{\text{odd}}(y) = a_1 + a_3y + \dots$

$$P(x) = P_{\text{even}}(x^2) + x \cdot P_{\text{odd}}(x^2)$$

The problem of size n (evaluating at x) is reduced to two problems of size $n/2$ (evaluating at x^2).

5.2.2 Roots of Unity

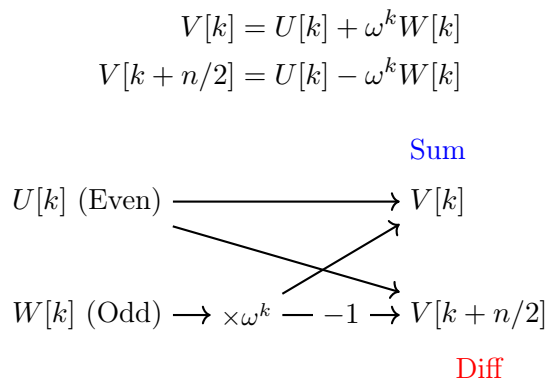
To make the recursion work, the points x must be chosen carefully so that squaring them reduces the set size. We use the **Complex n -th Roots of Unity**:

$$\omega_n = e^{2\pi i/n}$$

Properties: $\omega_n^n = 1$, and roughly half of the values are negatives of the others ($\omega_n^{k+n/2} = -\omega_n^k$).

5.2.3 The Butterfly Operation (Visualization)

In the recursive step, we combine results from the sub-problems (U and W) using the “Butterfly” structure.



5.3 FFT Algorithm Implementation

Based on the lecture notes, this recursive algorithm computes the Discrete Fourier Transform.

Algorithm 3 Recursive-FFT (a, n, ω)

Require: Coefficient vector $a = (a_0, \dots, a_{n-1})$, size n (power of 2), primitive root ω

Ensure: Vector V containing values of polynomial at roots of unity

```

1: if  $n = 1$  then
2:   return vector  $(a_0)$  ▷ Base case
3: end if
4:  $\omega_{sq} \leftarrow \omega^2$ 
5:  $a_{even} \leftarrow (a_0, a_2, \dots, a_{n-2})$ 
6:  $a_{odd} \leftarrow (a_1, a_3, \dots, a_{n-1})$ 
7:  $U \leftarrow \text{Recursive-FFT}(a_{even}, n/2, \omega_{sq})$  ▷ Recurse on Evens
8:  $W \leftarrow \text{Recursive-FFT}(a_{odd}, n/2, \omega_{sq})$  ▷ Recurse on Odds
9:  $\omega_{curr} \leftarrow 1$ 
10: for  $k = 0$  to  $n/2 - 1$  do
11:    $V[k] \leftarrow U[k] + \omega_{curr} \cdot W[k]$  ▷
12:    $V[k + n/2] \leftarrow U[k] - \omega_{curr} \cdot W[k]$  ▷
13:    $\omega_{curr} \leftarrow \omega_{curr} \cdot \omega$ 
14: end for
15: return  $V$ 

```

5.4 Inverse FFT and Complexity

- **Inverse FFT:** To convert back from Point-Value to Coefficient form, we use the same algorithm but replace ω with ω^{-1} (or $1/\omega$) and divide the final result by n .
- **Time Complexity:** The recurrence is $T(n) = 2T(n/2) + O(n)$.

$$T(n) = O(n \log n)$$

This is a significant improvement over the $O(n^2)$ naive multiplication.

6 Module 6: String Matching

Source: Lecture 22

Problem: Given a text T of length n and a pattern P of length m , find all occurrences of P in T .

6.1 Naive Approach

The naive approach shifts the pattern by 1 position each time a mismatch occurs.

- **Logic:** If $P[j] \neq T[i]$, reset $j = 0$ and move i back to $i - j + 1$.
- **Complexity:** $O(nm)$ in the worst case.

6.2 Knuth-Morris-Pratt (KMP) Algorithm

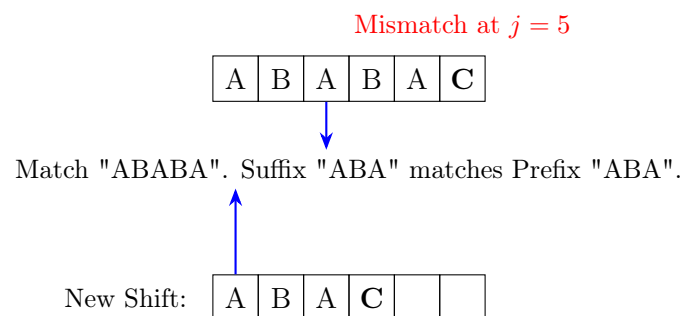
Deep Dive Integration

The KMP algorithm improves upon the naive approach by observing that when a mismatch occurs, the pattern itself contains enough information to determine where the next potential match could begin, avoiding redundant comparisons.

6.2.1 The Failure Function (Next Array)

The core of KMP is the preprocessing of Pattern P to create a **next** array (also known as the π table).

- **Definition:** $\pi[q]$ is the length of the longest proper prefix of $P[0 \dots q]$ that is also a suffix of $P[0 \dots q]$.
- **Insight:** If we mismatch at index j , we fall back to **next**[$j-1$] instead of 0.



6.2.2 KMP Algorithms

1. Computing the Next Array (Preprocessing)
2. The Matcher

6.2.3 Complexity Analysis

- **Preprocessing Time:** $O(m)$
- **Matching Time:** $O(n)$ (Amortized analysis shows i increases n times, and j decreases at most as many times as it increases).
- **Total Time:** $O(n + m)$
- **Space:** $O(m)$ for the array.

Algorithm 4 Compute-Next-Array (P)

```
1: Let  $m$  be length of  $P$ ,  $next[0] \leftarrow 0$ ,  $len \leftarrow 0$ ,  $i \leftarrow 1$ 
2: while  $i < m$  do
3:   if  $P[i] == P[len]$  then
4:      $len \leftarrow len + 1$ ;  $next[i] \leftarrow len$ ;  $i \leftarrow i + 1$ 
5:   else
6:     if  $len \neq 0$  then
7:        $len \leftarrow next[len - 1]$  ▷ Backtrack
8:     else
9:        $next[i] \leftarrow 0$ ;  $i \leftarrow i + 1$ 
10:    end if
11:  end if
12: end while
13: return  $next$ 
```

Algorithm 5 KMP-Matcher (T, P)

```
1:  $next \leftarrow \text{Compute-Next-Array}(P)$ 
2:  $i \leftarrow 0$  (Text Index),  $j \leftarrow 0$  (Pattern Index)
3: while  $i < n$  do
4:   if  $P[j] == T[i]$  then
5:      $j \leftarrow j + 1$ ;  $i \leftarrow i + 1$ 
6:   end if
7:   if  $j == m$  then
8:     Found at  $(i - j)$ ;  $j \leftarrow next[j - 1]$ 
9:   else if  $i < n$  and  $P[j] \neq T[i]$  then
10:    if  $j \neq 0$  then  $j \leftarrow next[j - 1]$ 
11:    else  $i \leftarrow i + 1$ 
12:    end if
13:  end if
14: end while
```
